High-Level Design Report

1. Introduction

1.1 Purpose of the system

The primary purpose of this system is to provide a unified, AI-driven Cyber Threat Intelligence (CTI) platform for Security Operations Centers (SOCs). Existing security operations are often fragmented, leading to "alert fatigue" where analysts are overwhelmed by raw data from disjointed tools. We aim to solve this by automating critical tasks such as malware classification, phishing detection, and intrusion detection, integrating them into a single dashboard to enhance situational awareness and decision-making efficiency.

1.2 Design goals

- **Modularity within Monolith:** While the system is a monolithic web application for simplified deployment, it is designed with distinct layers to ensure maintainability.
- **Responsiveness:** The system utilizes an event-driven architecture with background workers to handle heavy computations (like ML inference), ensuring the user interface remains responsive (e.g. IDS alerts displayed under 2 seconds).
- **Security & Isolation:** A core goal is the safe execution of dangerous tasks; therefore, malware analysis and attack simulations are strictly isolated in sandboxed environments to prevent host contamination.
- **Scalability:** The use of a message queue decouples data ingestion from processing, allowing the system to handle bursts of log data without crashing

1.3 Definitions, acronyms, and abbreviations

- **SOC:** Security Operations Center
- **IDS:** Intrusion Detection System
- **CVE:** Common Vulnerabilities and Exposures
- **RBAC:** Role-Based Access Control
- **KMS:** Key Management Service
- **ML:** Machine Learning

1.4 Overview

This document provides a high-level architectural view of the project. It details the decomposition of the system into functional subsystems, the mapping of software components to hardware units, data management strategies, and the control flow that governs system behavior

2. Current software architecture (if any)

There is no pre-existing software architecture for this project as it is a new development project.

3. Proposed software architecture

3.1 Overview

The proposed architecture is a Layered Monolith augmented by an Event-Driven Worker Subsystem.

- **Core Framework:** The application logic is built on Django (Python), serving as the central hub for API management and user interaction.
- **Asynchronous Processing:** To avoid blocking the main web server, heavy tasks (such as parsing logs or running AI models) are offloaded to a message queue (RabbitMQ/Redis) and processed by a pool of Celery workers.
- **Frontend:** The presentation layer utilizes a web dashboard (React/HTML) that communicates via REST APIs and WebSockets for real-time updates.

3.2 Subsystem decomposition

1. **Presentation Layer:**
    - **Dashboard UI:** Handles visualization of alerts, graphs, and reports.
    - **WebSocket Gateway:** Manages real-time data push to clients.
2. **Application/API Layer:**
    - **API Collector:** The entry point for external webhooks (IDS logs) and internal requests.
    - **Auth & User Management:** Handles login, RBAC, and user profiles.
3. **Service/Processing Layer (Worker Pool):**
    - **Normalizer Worker:** Converts raw logs into a unified schema.
    - **Inference Worker (ML Orchestrator):** Runs AI models for anomaly and phishing detection.
    - **Sandbox Manager:** Provisions isolated VMs for malware analysis and attack simulation.
    - **CVE Fetcher:** periodically pulls vulnerability data from external APIs.
4. **Data Layer:**
    - **Primary DB:** MySQL database for structured data.
    - **Cache/Queue:** Redis/RabbitMQ for session management and task queuing

3.3 Hardware/software mapping

Software Component	Deployment Unit	Underlying Technology
Web Backend	**Application Container**	**Docker, Python/Django**
Message Broker	**Queue Container**	**RabbitMQ or Redis**
Worker Nodes	**Worker Containers**	**Celery**
Sandbox Environment	**Virtual Machine / Isolated Container**	**Docker, CALDERA**
Database	**Database Container**	**MySQL Server**

3.4 Persistent data management

This projects uses a relational database (MySQL) as the primary persistence mechanism to ensure data integrity.

- **Structured Data:** Users, Blacklists, Normalized Alerts, CVEs, and Audit Logs are stored in normalized relational tables.
- **Unstructured Data:** Binary files (malware samples) and large report artifacts are stored in Object Storage with file paths referenced in the database.

3.5 Access control and security

- **RBAC:** The system implements Role-Based Access Control with distinct roles: Admin, Analyst, and ReadOnly.
- **Authentication:** Passwords are hashed using strong algorithms (Argon2/Bcrypt). Two-Factor Authentication (2FA) is mandatory for Admin roles to protect privileged operations.
- **Secret Management:** Sensitive keys (API tokens, database credentials) are managed via a Key Management Service (KMS) or secure vault, never stored in plain text.
- **Isolation:** The Sandbox Manager ensures that untrusted code execution (malware/simulation) occurs in strictly isolated environments with no network leakage

3.6 Global software control

The global control flow is Event-Driven and Asynchronous:

- **Ingestion:** External events (IDS logs, email uploads) hit the API Layer.
- **Queuing:** The API validates the request and immediately pushes a job to the Message Queue, returning a "Received" status to the client
- **Processing:** Background workers consume messages from the queue. The Scheduler coordinates execution order and retries.
- **Notification:** Upon task completion, workers update the Database and trigger the WebSocket Gateway to push the result to the connected Dashboard clients.

3.7 Boundary conditions

- **Startup:** The system initializes the DB connection and checks for the availability of the Message Queue before accepting traffic.
- **Graceful Degradation:** If the ML Inference Service fails or GPUs are unavailable, the system falls back to basic rule-based analysis or CPU processing, logging the error without crashing the entire platform.
- **Shutdown:** Workers complete currently running jobs before terminating to prevent data corruption.

4. Subsystem services

The major subsystems provide the following services:

- **Authentication Service:** Verifies credentials, issues session tokens, and validates TOTP codes.
- **Normalization Service:** Accepts raw JSON logs from disparate IDS vendors and converts them into the system's standard IDS alerting format.
- **Inference Service:** Provides an internal API to load ML models and return classification scores
- **Scan Service:** Orchestrates Nmap/OpenSSL tools to scan a domain and return a structured JSON report of open ports and certificate status.
- **Reporting Service:** Aggregates data from the database and renders HTML templates into downloadable PDF documents.

5. Glossary

- **Monolithic Architecture:** A software architecture where all components are interconnected and interdependent, rather than loosely coupled.
- **Worker:** A background process dedicated to executing asynchronous tasks popped from a queue.
- **Sandbox:** An isolated computing environment in which a program or file can be executed without affecting the application it runs on.
- **Webhook:** A method of augmenting or altering the behavior of a web page or web application with custom callbacks

6. **References**
    - **SRS: Software Requirements Specification v2.0.**
    - **SDD: Software Design Description v1.1.**
    - **PMP: Project Management Plan v2.0.**
    - **STP: Software Test Plan v1.0.**
    - **QMP: Quality Management Plan v1.0.**